

Guide Complet : Utiliser le Design System avec Cursor

Table des Matières

1. Préparation
 2. Configuration du Projet
 3. Utilisation dans Cursor
 4. Exemples Pratiques
 5. Conseils Pro
-

1 Préparation {#préparation}

Étape 1 : Sauvegarder le Design System

1. **Copiez le JSON** que j'ai créé
2. **Créez un fichier** dans votre projet :

```
votre-projet/  
└─ design-system.json
```

3. **Collez le contenu** du JSON dans ce fichier

Étape 2 : Installer Cursor

- Téléchargez Cursor depuis cursor.sh

- Installez-le sur votre machine
 - Ouvrez votre projet dedans
-

2 Configuration du Projet {#configuration}

Structure Recommandée

```
mon-restaurant-app/  
├── design-system.json    # ← Le fichier JSON  
├── src/  
│   ├── components/      # Composants React/Vue  
│   ├── styles/          # Fichiers CSS/SCSS  
│   ├── pages/           # Pages de l'app  
│   └── utils/           # Utilitaires  
├── android/             # Pour Android  
├── ios/                 # Pour iOS/macOS  
└── README.md
```

3 Utilisation dans Cursor {#utilisation}

Méthode 1 : Chat avec Contexte (@-mention)

1. **Ouvrez le Chat Cursor** (Cmd/Ctrl + L)
2. **Mentionnez le fichier** avec `@`
3. **Donnez vos instructions**

Exemple :

@design-system.json Crée une page d'accueil pour un restaurant avec un hero section et une grille de menu items. Utilise la palette "japanese" et le layout "hero".

Méthode 2 : Composer (pour générer du code)

1. **Ouvrez Composer** (Cmd/Ctrl + I)
2. **Ajoutez le design system en contexte**
3. **Demandez la génération de code**

Exemple :

En utilisant le design system dans design-system.json :

1. Crée un composant React "MenuCard"
2. Utilise les styles de composants.card.menuItem
3. Applique la palette de couleurs "vegetarian"
4. Ajoute les animations hover définies

Méthode 3 : Rules for AI (.cursorrules)

Créez un fichier `.cursorrules` à la racine :

Rules for AI

Design System

Always reference design-system.json for:

- Colors (use colorPalettes)
- Typography (use typography.scale)
- Spacing (use spacing.scale)
- Components (use components)

Code Style

- Use TypeScript
- Follow the component structure defined
- Apply responsive breakpoints
- Include accessibility features

Naming Conventions

- Components: PascalCase
- Files: kebab-case
- CSS Classes: BEM notation

4 Exemples Pratiques {#exemples}

Exemple 1 : Créer une Page Menu (Web - React)

Prompt dans Cursor :

@design-system.json

Crée une page Menu pour restaurant avec :

- Header sticky (components.navigation.header)
- Grille de menu items (layoutPatterns.menuGrid)
- Cards pour chaque plat (components.card.menuItem)
- Palette de couleurs "japanese"
- Animations hover
- Responsive pour mobile/tablette/desktop

Stack : React + TypeScript + Tailwind CSS

Exemple 2 : Créer un Formulaire de Réservation

Prompt dans Cursor :

@design-system.json

Génère un composant ReservationForm avec :

- Tous les champs de features.reservation.fields
- Styles de components.form
- Validation selon features.reservation.validation
- Bouton primary avec animation
- Messages d'erreur avec colors.semantics.error

Stack : React + TypeScript

Exemple 3 : Application Android

Prompt dans Cursor :

@design-system.json

Crée une activité MenuActivity pour Android avec :

- Bottom Navigation (platforms.android.adaptations.navigation)
- RecyclerView avec des MenuItemCards
- Material Design selon components.card.menuItem
- Palette "fastfood"

Stack : Kotlin + Jetpack Compose

Exemple 4 : Application macOS

Prompt dans Cursor :

@design-system.json

Crée une vue MenuView pour macOS avec :

- Sidebar Navigation (platforms.macos.adaptations.navigation)
- Grid de plats
- SwiftUI selon platforms.macos.framework
- Palette "vegetarian"

Stack : SwiftUI

5 Conseils Pro {#conseils}

✓ Bonnes Pratiques

1. Soyez Spécifique

✗ "Crée une page menu"

✓ "Crée une page menu avec `layoutPatterns.menuGrid`,
utilise `components.card.menuItem` et la palette `japanese`"

2. Référencez les Chemins JSON

Utilise :

- `colorPalettes.japanese.primary`
- `typography.scale.h1`
- `spacing.layout.sectionPadding`

3. Demandez du Code Adaptatif

Génère le code responsive selon :

- `responsive.breakpoints`
- `responsive.containers`

4. Incluez l'Accessibilité

Respecte `accessibility.contrast` et
`accessibility.keyboardNavigation`

Prompts Multi-Étapes

Pour un Projet Complet :

Étape 1 - Structure :

@design-system.json

Crée la structure de base du projet React avec :

- Configuration Tailwind utilisant les couleurs du design system
- Fichiers de thème globaux
- Structure de dossiers selon les composants définis

Étape 2 - Composants de Base :

@design-system.json

Génère les composants de base :

1. Button (toutes les variantes)
2. Card (menuItem et reservation)
3. Input, Select, Textarea

Avec TypeScript et Props définies

Étape 3 - Pages :

@design-system.json

Crée la HomePage avec :

- Hero section (layoutPatterns.hero)
- Section "À Propos" (layoutPatterns.about)
- Footer (layoutPatterns.footer)

Étape 4 - Fonctionnalités :

@design-system.json

Implémente :

- Système de menu avec filtres (features.menu)
- Formulaire de réservation (features.reservation)
- Panier de commande (features.onlineOrdering)

🔧 Personnalisation

Pour Modifier le Design System :

1. Changer les Couleurs :

@design-system.json

Modifie la palette "japanese" :

- Primary: #2C3E50
- Secondary: #E74C3C

Puis régénère le composant Header

2. Ajouter un Nouveau Thème :

@design-system.json

Ajoute une nouvelle palette "italian" avec :

- Primary: #C41E3A (rouge italien)
- Secondary: #009246 (vert italien)
- Accent: #FFFFFF

Puis crée une page avec ce thème

Pour Chaque Plateforme

Web (React) :

@design-system.json

Stack : React 18 + TypeScript + Tailwind CSS

Crée [votre composant]

Android :

@design-system.json

Stack : Kotlin + Jetpack Compose

Utilise platforms.android.adaptations

Crée [votre composant]

macOS :

@design-system.json

Stack : SwiftUI

Utilise platforms.macos.adaptations

Crée [votre composant]

Exercices Pratiques

Exercice 1 : Page de Menu Complète

@design-system.json

Crée une page MenuPage.tsx avec :

1. Header avec logo et navigation
2. Section hero avec image de fond
3. Filtres de catégories
4. Grille de 12 plats (layoutPatterns.menuGrid)
5. Chaque carte utilise components.card.menuItem
6. Footer complet
7. Entièrement responsive
8. Animations au scroll

Exercice 2 : Système de Thèmes

@design-system.json

Crée un système de switch entre thèmes :

1. ThemeProvider React Context
2. Toggle entre "japanese", "mexican", "vegetarian"
3. Applique les couleurs dynamiquement
4. Sauvegarde le choix dans localStorage

Exercice 3 : Application Mobile Complète

@design-system.json

Pour Android, crée une app complète avec :

1. SplashScreen
2. MenuActivity avec Bottom Nav
3. ReservationActivity
4. CartActivity

5. ProfileActivity

Utilise Jetpack Compose et Material Design

Debugging

Si Cursor ne comprend pas votre design system :

1. **Vérifiez le JSON** : Il doit être valide
2. **Simplifiez** : Demandez un composant à la fois
3. **Soyez explicite** : Indiquez les chemins JSON exacts
4. **Itérez** : Demandez des modifications progressives

Exemple de Debug :

Le composant ne respecte pas les couleurs.

Vérifie que tu utilises bien :

- colorPalettes.japanese.primary pour le bouton
- colorPalettes.japanese.text.primary pour le texte

Régénère le composant Button

Vous êtes Prêt !

Maintenant vous pouvez :

1.  Utiliser le design system dans Cursor
2.  Générer des composants cohérents

3. ☒ Créer des apps Web, Android et macOS
4. ☒ Maintenir une cohérence visuelle

Commencez par un petit composant et progressez !